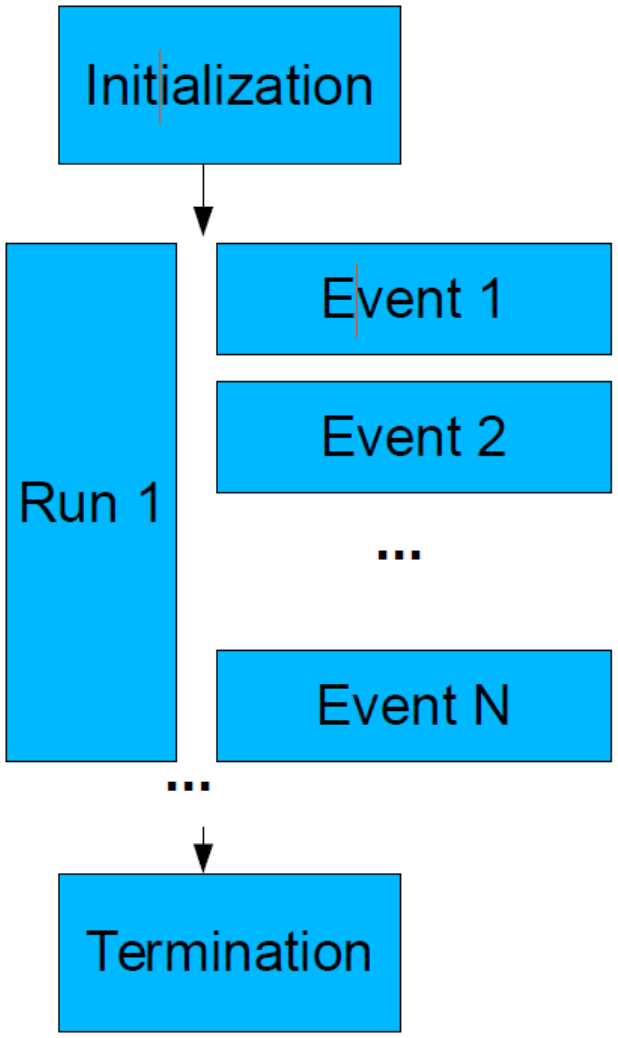
Цикл моделирования



Необходимые параметры для запуска

- Распределение вещества в детекторе и поля
- Генератор первичной вершины
- Список физических процессов, учитываемых в моделировании

Дополнительные настройки

- Чувствительные элементы и способы моделирования отклика
- Способы визуализации
- Графический интерфейс
- Пользовательские расширения

Самый главный класс Geant4

Под названием Geant4 скрываются сотни классов, решающих ту или иную задачу. Однако связь между собой имеет лишь ограниченное число, базовых, а часто абстрактных базовых классов, вместе образующих ядро Geant4. Для инициализации ядра в Geant4 существует специальный класс G4RunManager. Этот класс является управляющим. Он контролирует последовательность выполнения программы, а также управляет циклом событий во время запуска. Кроме того, пользователь может использовать иную версию данного класса — G4MTRunManager, если работает в многопоточном режиме.

Выделение места в динамической памяти для объекта класса G4RunManager и возврат указателя на созданный объект:

```
G4RunManager* runManager = new G4RunManager;
```

одной из функций runManager является связь пользовательских классов с ядром Geant4

для корректной работы любого приложения необходимо передать информацию:

ГДЕ ПРОИСХОДИТ МОДЕЛИРОВАНИЕ?

ПО КАКИМ ЗАКОНАМ ПРОИСХОДИТ МОДЕЛИРОВАНИЕ?

ЧТО ПОЛЬЗОВАТЕЛЬ ХОЧЕТ УЗНАТЬ О РЕЗУЛЬТАТАХ?

На первый вопрос отвечает класс геометрии

На второй — класс, содержащий список всех физических процессов, которые могут произойти в рамках данного моделирования

на третий вопрос отвечает класс пользовательских действий, включающий информацию о том, какие первичные частицы рождаются

Класс G4RunManager содержит метод **G4RunManger**::**SetUserInitialization**(), который позволяет инициализировать и связывать с ядром разные части процесса моделирования (*геометрия, описание физических процессов или классы действий*) .

```
runManager->SetUserInitialization(new Geometry);

runManager->SetUserInitialization(physicsList);

runManager->SetUserInitialization(new Action());
```

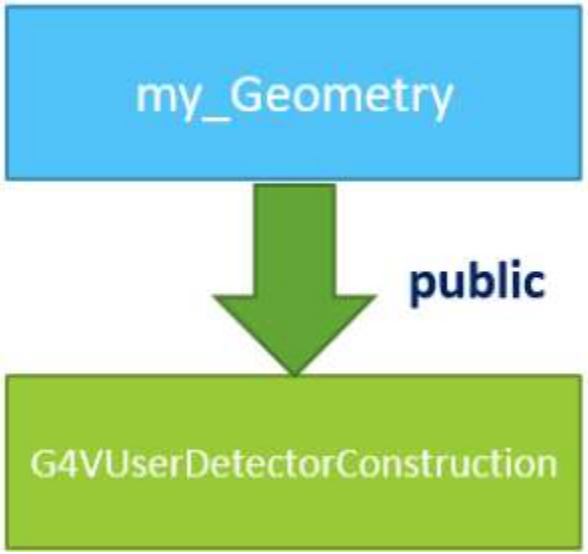
После того как вся необходимая информация передана, вызывается метод G4RunManager::Initialize()

```
runManager->Initialize();
```

runManager->SetUserInitialization(new Geometry);

G4VuserDetectorConstruction базовый абстрактный класс для любой пользовательской геометрии. Реализация потомка данного класса являет обязательным этапом в каждом проекте моделирования.

Класс Geometry является потомком класса G4UserDetectorConstruction



G4VUserDetectorConstruction обладает единственным чисто виртуальным методом

Construct()

Данный метод вызывается объектом класса **G4RunManager** во время инициализации

то есть, *во время вызова метода* **G4RunManager**::**Initialize**()

Построение модели детектора

Модель детектора строится из простых элементов – объемов

- Описание материалов
- Описание объемов
- *Описание электромагнитных полей*
- *Свойства визуализации*
- *Детектирующие свойства объемов*

Классы, описывающие материалы

Материал для Geant4 = количество атомов данного вида на единицу объема (длины пробега)

- **G4Isotope**

  описывает свойства атома: атомное число, количество нуклонов, молярную массу и т.д.

- **G4Element**

  описывает свойства элемента: эффективное атомное число, эффективную молярную массу, число изотопов

- **G4Material**

  описывает макроскопические свойства вещества:

  плотность, состояние, температуру, давление, радиационную длину, длину свободного пробега и т.д

Для создания объёма детектора нужно определить материал

Структура материалов в Geant4 отражает то, что реально существует в природе: материалы состоят из отдельных элементов или их смесей, элементы в свою очередь состоят из отдельных изотопов или их смесей.

Создание элемента из изотопов

обязательными полями являются: имя, атомный номер и количество нуклонов. если не указана, то молярная масса рассчитается автоматически

G4int iz,n,ncomponents;
G4double a;
G4String name, symbol;

```
G4Isotope* U5 = new G4Isotope(name="U235", iz=92, n=235, a=235.01*g/mole);
G4Isotope* U8 = new G4Isotope(name="U238", iz=92, n=238, a=238.03*g/mole);
G4Element* elU  = new G4Element(name="enriched Uranium", symbol="U",  ncomponents=2);

elU->AddIsotope(U5, abundance= 90.*perCent);
elU->AddIsotope(U8, abundance= 10.*perCent);
```

Для каждого создаваемого элемента данный метод должен вызываться ровно столько раз, сколько изотопов было указано в конструкторе.

Создание материала:

```
density=19*g/cm2;
G4Material* matU = new G4Material(name="matU", density, ncomponents=1);
matU→AddElement(elU, natoms=1);
```

База материалов National Institute of Standards and Technology (NIST)

В Geant4 представлена база материалов, составленная NIST Physical Measurement Laboratory. В данной базе доступно более 3000 изотопов, а все представленные элементы составлены исходя из природного баланса изотопов. Элементы доступны от Водорода до Калифорния (98). Кроме того, в базе доступны различные готовые материалы, такие как:

- составные вещества и смеси (ткань, эквивалентная пластику; морской воздух и т.д.)
- биохимические материалы (жировая ткань, цитозин, тимин и т.д.)
- композитные материалы (например, кевлар)

Как посмотреть таблицу изотопов, элементов и материалов

```
G4cout << *(G4Isotope::GetIsotopeTable()) << G4endl;
G4cout << *(G4Element::GetElementTable()) << G4endl;
G4cout << *(G4Material::GetMaterialTable()) <<G4endl;
```

Перенаправление стандартного вывода и стандартного вывода ошибок в файл: ./TrackStack > file.dat 2>&1

Библиотека материалов Geant4

```
#include "G4NistManager.hh"

 G4NistManager* man = G4NistManager::Instance();

// define elements
  G4Element* elAl = man->FindOrBuildElement("Al");

// define pure NIST materials

G4Material* Al = man->FindOrBuildMaterial("G4_Al");
G4Material* Cu = man->FindOrBuildMaterial("G4_Cu");

// define NIST materials

G4Material* H2O  = man->FindOrBuildMaterial("G4_WATER");

G4Material* Sci  = man-> FindOrBuildMaterial("G4_PLASTIC_SC_VINYLTOLUENE");

G4Material* Vacuum  = man-> FindOrBuildMaterial("G4_Galactic");
```

Полный список доступных материалов в Geant4 можно найти в разделе:

Geant4 Book For Application Developers > Appendix > Geant4 Material Database

Geant4

типы переменных

ввод-вывод

библиотека CLHEP

система единиц

Основные типы переменных Geant4

Для достижения переносимости кода в Geant4 переопределены основные типы переменных

G4int, *G4long*, *G4float*, *G4double*,

G4bool, *G4complex*, *G4String*

Ввод и вывод в коде Geant4

Можно использовать обычные printf() и cout

Однако более правильно использовать переопределенные потоки Geant4:

G4cout << *“test”* << *G4endl*;

G4cerr << *“error”* << *G4endl*;

Система единиц Geant4

Для некоторых единиц измерения в Geant4 установлено значение «1».

- millimeter (mm) - миллиметр
- nanosecond (ns) - наносекунда
- MeV - Мега электронвольт
- eplus - заряд позитрона
- kelvin - кельвин
- mole - моль
- candela -сила света
- radian - радиан
- steradian - стерадиан

Все остальные единицы задаются через указанные в этом списке величины

единицы измерения описаны в файле G4SystemOfUnits.hh.

Любое число, имеющее размерность, должно быть умножено на соответствующую единицу для перевода во внутреннюю систему единиц Geant4

length = 10.0 * cm;
kinetic_energy = 5.0 * GeV;

```
double length = 5. * cm;
G4cout << length << '\n';
```

Выводимым в консоль результатом будет 50.

Для вывода величины в желаемых единицах следует делить число на единицу измерения

G4cout << eDep / MeV << " [MeV]" << G4endl;

```
double mass = 5. * g;
G4cout << mass / g << '\n';
```

Выводимый результат будет 5

если существует необходимость в добавлении единиц измерения, то можно добавить в файл с описанием геометрии строки

```
#include <G4SystemOfUnits.hh>

static const G4double inch = 2.54*cm;
```

чтобы увидеть весь список доступных единиц измерения, можно воспользоваться командой:

/units/list (например, воспользоваться списком команд в меню визуализатора)

Геометрия.

В основе геометрии Geant4 лежит концепция логических и физических объемов. Логический объем представляет собой элемент детектора определенной формы заданного материала, чувствительности и т. п. Кроме того, каждый логический объем может содержать внутри другие объемы. Физический объем представляет собой пространственное расположение логического объема относительно логического материнского объема

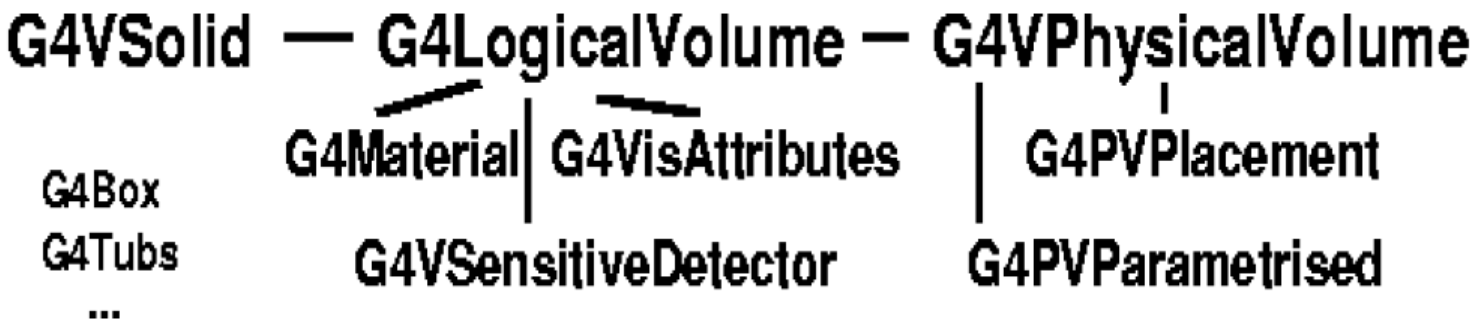
Объем описывается в три этапа

- форма (G4VSolid)
- логический объем (G4LogicalVolume)
- физический объем (G4VPhysicalVolume)

- На первом этапе создается основа будущего логического объема - его форма, включающая геометрические характеристики.

- На втором - описываются свойства формы, такие как: материал, сцинтилляционные свойства и т. п. Кроме того, на втором этапе форме можно установить различные визуальные атрибуты, упрощающие дальнейшую визуализацию моделирования.

- На третьем этапе осуществляется расположение модифицированной формы в пространстве, её поворот, смещение и т. п.



в Geant4 представлен целый набор примитивных форм, являющихся потомками абстрактного класса G4SCGSolid : коробки, сферы, конусы, трубки и др.

Простейшая форма: прямоугольный параллелепипед (коробка):

```
G4Box(const G4String& pName, G4double pX, G4double pY, G4double pZ);
```

в качестве значений по X, Y, Z принимаются ПОЛОВИНЫ от реальной длины, ширины и высоты. Данное свойство характерно для большинства форм, используемых в G4.

реализация объекта простейшей формы будет выглядеть следующим образом:

```
G4Box *box = new G4Box("box", 5*cm, 5*cm, 5*cm);
```

Все геометрические формы в Geant4 реализованы в соответствии с концепцией **Конструктивной блочной геометрии** Constructive Solid Geometry (CSG). Большинство комплексных форм описывается за счет их граничной поверхности, которая может быть первого, второго порядка, или B-spline поверхностью. Все это сделано, опираясь на ISO STEP стандарт CAD систем.

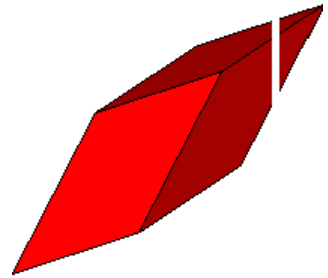
Простейшие тела, используемые в конструктивной блочной геометрии — **примитивы** (англ. *primitives*), тела с простой формой:

куб, цилиндр, призма, пирамида, сфера, конус.

Построение более сложного объекта происходит путём применения к описаниям объектов булевых (двоичных) операций на множествах — объединение, пересечение и разность.

Формы

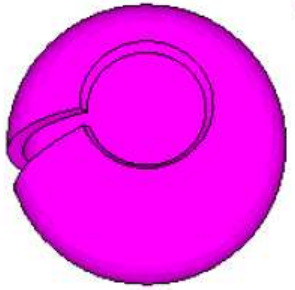
Geant4 Book For Application Developers > Geometry





Простые формы

G4Box, G4Tubs, G4Cons, G4Trd, ...

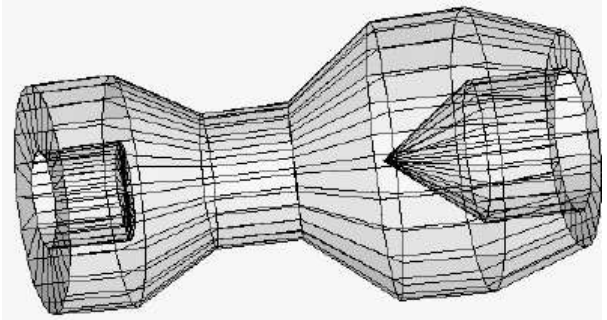


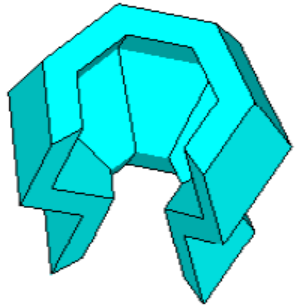
Специальные формы

G4polycone, G4Polyhedra, G4Hype, ...

Определяемые поверхностью

G4BREPSolidPolycone, G4BSplineSurface, ...

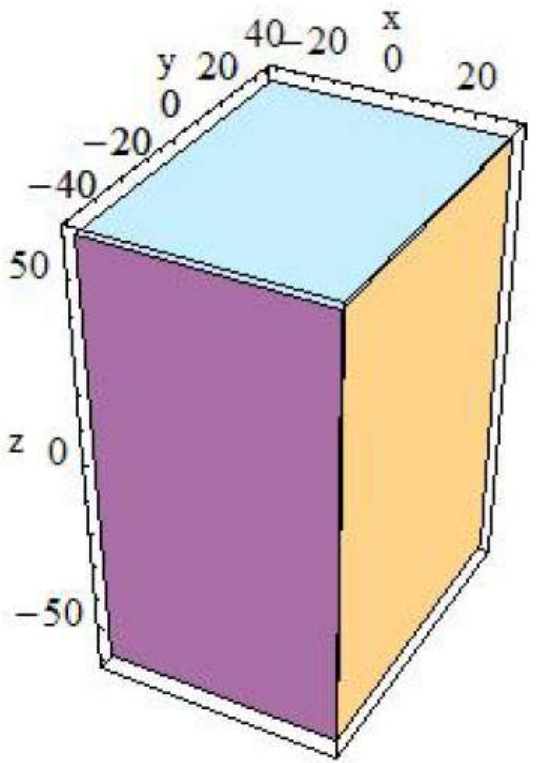




Булевы формы

G4UnionSolid, G4SubtractionSolid, ...

Параллелепипед



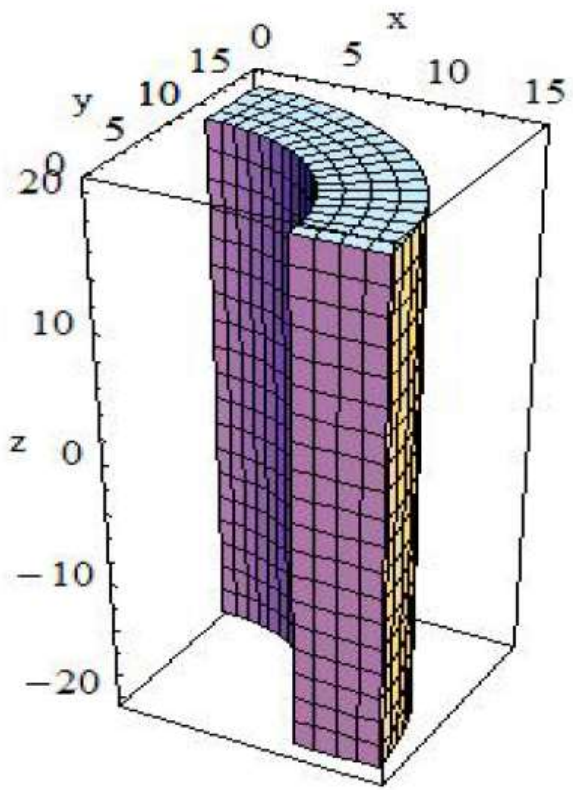

```
G4VSolid* scint_solid = new
   G4Box(const G4String& pName,
                G4double  pX,
                G4double  pY,
                G4double  pZ);
```

Пример:

```
G4Box* aBox = new
G4Box("BoxA", 20.0*cm, 40.0*cm, 60.0*cm);
```

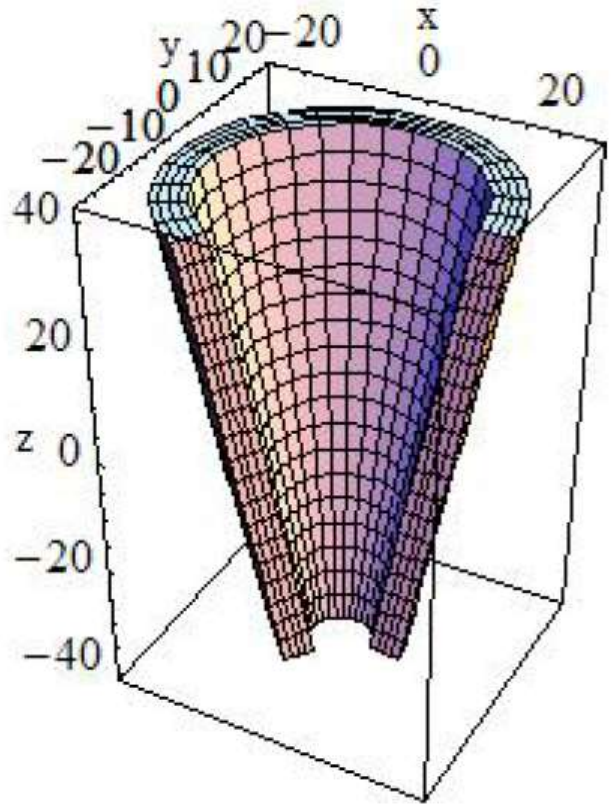
Цилиндр

```
G4VSolid* calor_solid = new
   G4Tubs(const G4String& pName,
          G4double  pRMin,
          G4double  pRMax,
          G4double  pDz,    - полувысота
          G4double  pSPhi,
          G4double  pDPhi)
```



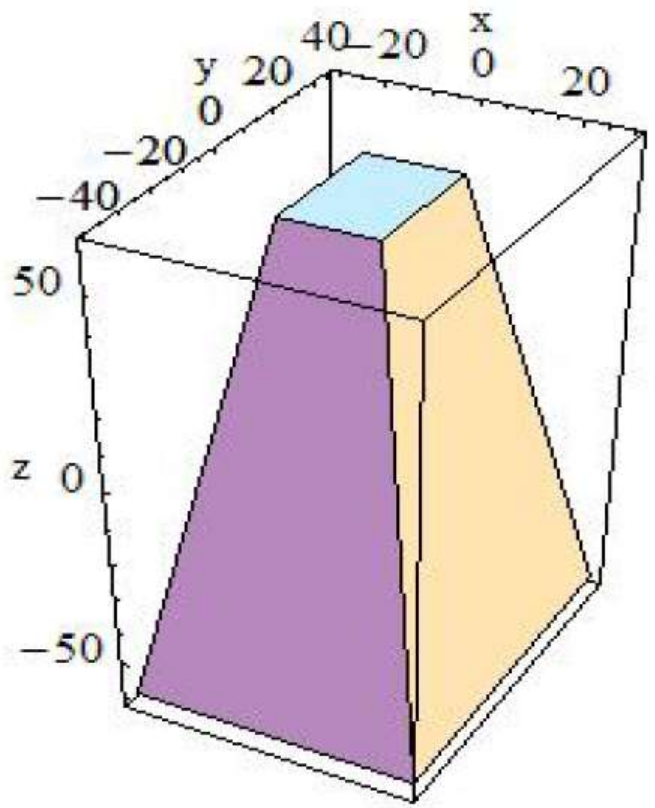
Конус

```
G4VSolid* scint_solid = new
 G4Cons(const G4String& pName,
        G4double  pRmin1,
        G4double  pRmax1,
        G4double  pRmin2,
        G4double  pRmax2,
        G4double  pDz,
        G4double  pSPhi,
        G4double  pDPhi)
```



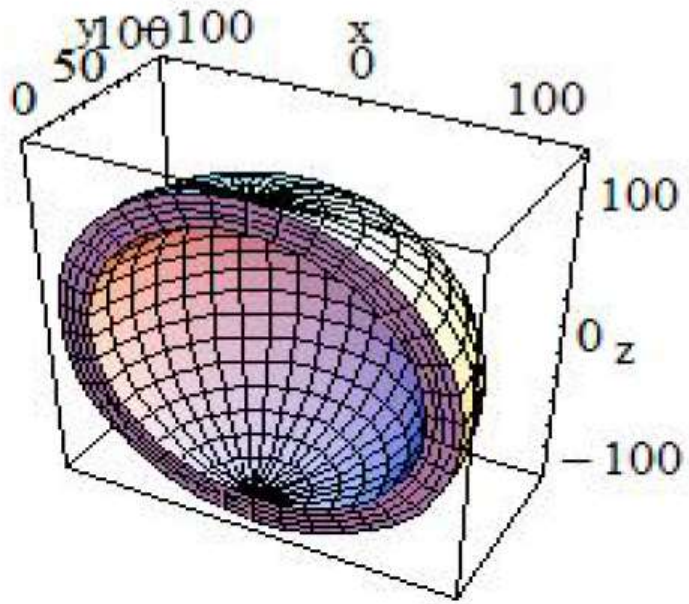
Трапезоид

```
G4VSolid* aSolid = new
 G4Trd(const G4String& pName,
             G4double  dx1,
             G4double  dx2,
             G4double  dy1,
             G4double  dy2,
             G4double  dz)
```



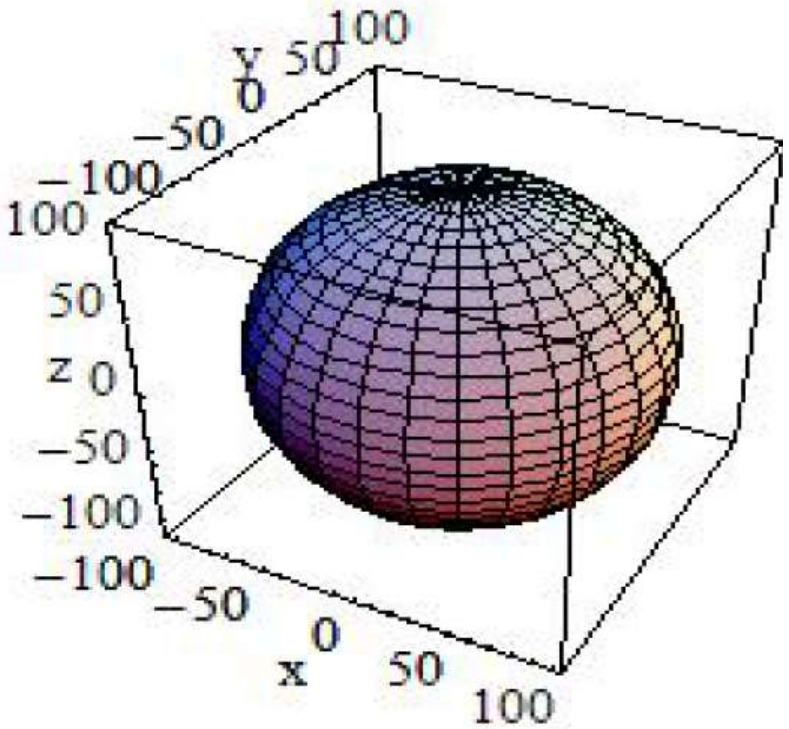
Сфера

```
G4VSolid* aSolid = new
  G4Sphere(const G4String& pName,
           G4double  pRmin,
           G4double  pRmax,
           G4double  pSPhi,
           G4double  pDPhi,
           G4double  pSTheta,
           G4double  pDTheta )
```



Шар

```
G4VSolid* aSolid = new
  G4Orb(const G4String& pName,
        G4double pRmax)
```



Логический объем

Сама по себе форма не содержит никакой информации об объекте, кроме его геометрических размеров. Однако для моделирования взаимодействия излучения с веществом необходимо описать свойства этого самого вещества, в котором осуществляется моделирование. С этой целью в Geant4 предоставлен класс объектов, называемый логический объем. Одной из основных целей логического объема является связь формы с физическими свойствами объекта, как пример, материал, из которого она(форма) сделана.

Объекты логического объема относятся к классу G4LogicalVolume.

Конструктор данного класса выглядит следующим образом:

```
G4LogicalVolume* aLogical =
        new  G4LogicalVolume( G4VSolid* pSolid,
            G4Material*           pMaterial,
            const G4String&       Name,
            G4FieldManager*       pFieldMgr=0,
            G4VSensitiveDetector* pSDetector=0,
            G4UserLimits*         pULimits=0,
            G4bool                optimise=true );
```

поля «форма и материал» (pSolid and pMaterial) не должны быть равными **nullptr**.

Поля: pFieldMgr, pSDetector, pULimits являются опциональными

Рассматривая формы, следует обратить внимание на визуализацию геометрических объектов, например цвет или прозрачность, так как визуальные атрибуты являются частью именно логической формы

Для того, чтобы изменить цвет того или иного геометрического объекта, следует вызвать метод:

```
void SetVisAttributes (const G4VisAttributes& VA);
```

который в качестве аргумента принимает интересующий нас параметр визуализации.

```
G4double box_size = 5 * cm;
G4Box *box = new G4Box("box", box_size, box_size, box_size);
G4LogicalVolume* box_log = new G4LogicalVolume(box, nist->FindOrBuildMaterial("G4_SODIUM_IODIDE"), "box_LOG");
box_log->SetVisAttributes(G4Colour::Blue());
```

Объект box_log представляет собой логический объем, в качестве аргументов - принимающий форму и моделируемый материал. Кроме того, осуществляется передача логическому объему визуального атрибута «синий цвет». Стоит отметить, что теперь все физические объёмы которые будут использовать в качестве параметра данный логический объем, окрасятся синим цветом.

Отдельно стоит отметить имена форм и логических объемов. В рамках объектов своего класса эти имена должны быть уникальны. Для формы, логического объема и физического объема одного геометрического объекта можно использовать одно имя.

Расчёт массы:

```
G4cout << "Mass = " << box_log->GetMass() / g << " g\n";
```

Физический объем

Последним этапом построения геометрического объекта средствами Geant4 является реализация его физического объема или, иначе говоря, расположение в пространстве.

- Строится на основе логического объема
- Описывает положение объема в пространстве
- Позволяет одновременно описать серию одинаковых объемов или параметризовать свойства объема в зависимости от номера копии

Вложенность объемов

- Все объемы должны быть вложены один в другой
  Перекрытие объемов не допускается!
- В любой модели существует только один “самый верхний объем” (экспериментальный зал), в который “вкладываются” все остальные
- Дочерний объем позиционируется в локальной системе координат, связанной с родительским объемом. Положение любого объекта (объема, частицы и т.д.) одновременно вычисляется как в глобальной координатной системе, связанной с экспериментальным залом, так и в локальной, связанной с объемом, в котором объект в данный момент находится

Размещение объёма

G4PVPlacement = один объем

Единственная копия данного объема размещается в материнском объеме

```
G4PVPlacement(G4RotationMatrix *pRot,          //матрица поворота
              const G4ThreeVector &tlate,      //вектор смещения
              G4LogicalVolume *pCurrentLogical,//логический объем
              const G4String& pName,           //имя
              G4LogicalVolume *pMotherLogical, //материнский объем
              G4bool pMany,                    //не используется
              G4int  pCopyNo,                  //номер копии
              G4bool pSurfChk=false);          //проверка на
                                               //пересечение с другими объемами
```

где pRot — матрица поворота, tlate — вектор смещения, pCurrentLogical — логический объем, pName — имя (идентификатор), pMotherLogical — материнский объем, pMany — данный параметр не реализован в текущей версии Geant4 (можно использовать false), pCopyNo - номер копии логического объема, для первого физического объема следует использовать значение 0, затем 1 и т.д.

должен существовать один единственный материнский логический объем, который становится изначальным или главным для всех последующих объемов.

```
world_pvpl = new G4PVPlacement(0, G4ThreeVector(0,0,0), world_log, "world_pvpl", 0, false, 0);
```

такой объем называют миром.

В качестве указателя на материнский объем для этого физического объема следует использовать нулевой указатель.

единственный необходимый для реализации геометрии метод: `virtual G4VPhysicalVolume* Construct() = 0;`

возвращает указатель на физический объем. Объем, на который возвращается указатель, является главным материнским объемом или представленным выше миром, с привязанными к нему всеми дочерними объемами, используемыми в геометрии.

Как задать матрицу поворота?

```
G4RotationMatrix* rotm = new G4RotationMatrix();
rotm->rotateX(90.*deg);
rotm->rotateZ(45.*deg);
```

Как задать трансляцию?

```
G4double pos_x = -1.0*cm;
G4double pos_y =  0.0*cm;
G4double pos_z =  0.0*cm;
G4ThreeVector(pos_x, pos_y, pos_z);
```

Матрица поворота: ZERO_RM, трансляция (смещение): DE1_vect

```
DE1_pvpl = new G4PVPlacement(ZERO_RM, DE1_vect, DE1_log, "DetEl_pvpl", world_log, 0, false, 0);
```